

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	ITA03	Course Title:	Mobile computing
CO Assessed:	CO4	Assessment:	AT2 – CASE STUDY
Weightage:	30 % of CIA	Total Marks:	100
CO4 Statement:	<i>Identify the components and structure of mobile application for Android and Windows OS-based devices.</i>		
Student Name:	I V Gowtham Kumar Reddy	Reg. No.:	192372158
Section / Batch:	CSE(AI)	Date:	18/08/2026

Code of Conduct

I I Gowtham Reddy (192372158) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: igowthamreddy

Instructions

- Duration: 60 minutes
- Number of questions : 01

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. A multimedia application performs intensive image processing on a smartphone. The application runs correctly but has poor performance when implemented entirely using high-level application APIs. Analyze how the Android runtime and native-level programming layers interact and determine how native programming could improve the application's performance.

REPORT

Introduction

A multimedia application performs intensive image processing on a smartphone. When the application is implemented entirely using high-level application APIs, it may experience poor performance because these APIs provide abstraction between the application and the lower system layers. For computationally intensive image processing, reducing unnecessary execution overhead can improve responsiveness and processing speed.

Android Runtime and Native-Level Programming

Android applications normally use the application framework and Android runtime to access system services and perform application operations. This approach simplifies development and provides a safer, more portable programming environment. However, intensive image-processing operations may require repeated high-level API calls and managed-runtime processing, which can introduce additional overhead.

Role of Native Programming

Native-level programming allows performance-critical parts of an application to be implemented closer to the system's native libraries and hardware capabilities. Image-processing operations such as filtering, pixel manipulation, transformation, compression, and other computationally intensive tasks can benefit from optimized native implementations. This can reduce runtime overhead and improve execution speed.

Interaction with Android Layers

The Android application framework provides high-level APIs, while the Android runtime manages application execution. Native libraries provide lower-level functionality, and the Linux kernel manages processes, memory, and hardware resources. Native programming can therefore shorten the execution path for performance-critical operations, while the operating system continues to control system resources and hardware access.

Conclusion

For an intensive multimedia application, implementing every operation only through high-level APIs is not always the best choice. A suitable approach is to keep normal application logic at the high level and move only computationally intensive image-processing components to native code. This can reduce runtime overhead and improve performance, responsiveness, and efficiency.